

자료구조 및 알고리즘

백준 77

Chapter 2. 재귀와 귀납적 사고

재귀 $\longleftrightarrow$ 귀납은 서로 바꿀 수 있다.

2.1 자료구조와 재귀

2.2 재귀 구조 예

2.3 재귀와 수학적 귀납법

- 재귀 (피보나치, 공차수열, 하노이탑, 팩토리얼)
- 선택정렬
- 중위, 전위, 후기 표기법
- 깊이우선탐색 (DFS)
- 이진탐색
- 재귀 & 수학적귀납법.



서열/회전/컨디션 매핑.

1. 자료구조와 재귀

- “내” 안의 “나”

- 동일한 성격에 크기만 작은 나를 찾아 연결된 관계를 찾는 것.
- 성격은 같고 크기만 작은 나를 찾아 큰 나와 작은 내가 연결된 관계를 드러내는 것



- **재귀 알고리즘 = 자기호출 알고리즘**

- 자신과 성격은 똑같지만 크기만 작은 알고리즘(들)을 호출하는 알고리즘
- 복잡한 문제도 간명하게 볼 수 있게 한다
- 예) 탐색, 정렬, 수열, ...
- 잘 쓰면 보약
 - 정렬, 탐색, ...
- 잘못 쓰면 독약
 - 피보나치수 구하기, 최적 행렬곱 경로, ...

2. 재귀 구조 예

- 수열
 - 등차 수열

$$a_n = a_{n-1} + 3, a_1 = 1$$

```
seq(n):  
    if (n = 1)  
        return 1  
    else  
        return seq(n-1) + 3
```

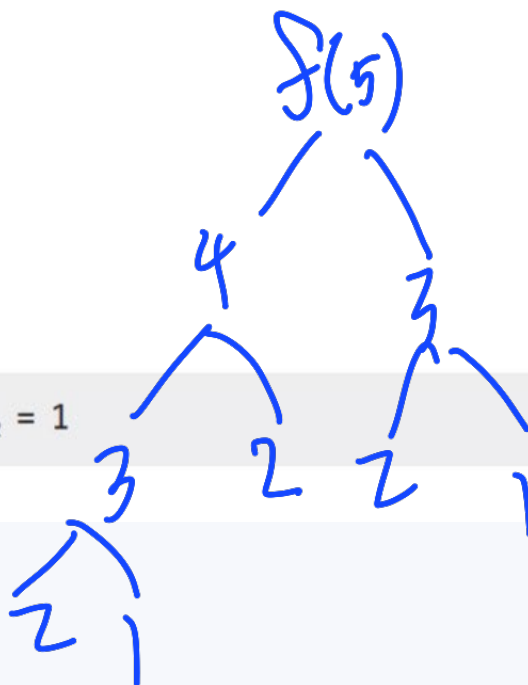
수열

피보나치 수열

$$f_n = f_{n-1} + f_{n-2}, f_1 = f_2 = 1$$

```
fib(n):  
  if (n=1 or n=2)  
    return 1  
  else  
    return fib(n-1) + fib(n-2)
```

```
fib_fast(n):  
  f[1] ← f[2] ← 1 ◀ "f[2] ← 1"과 "f[1] ← 1"을 한꺼번에 적어놓은 것  
  for i ← 3 to n  
    f[i] ← f[i-1] + f[i-2]  
  return f[n]
```



← 재귀

중복 호출로 인해 비효율적

← 비재귀

- 수열

- 피보나치 수열

fib(50) – 36초

fib(66) – 하루 정도

fib(100) – 3만 5천년정도

fib(136) – 1조년 정도

지수함수적 중복 호출로 인해 치명적인 비효율 발생!

- 수열

- 피보나치 수열

fib(n) :

$t[1] \leftarrow t[2] \leftarrow 1$

for $i \leftarrow 3$ **to** n

$t[i] \leftarrow t[i-1] + t[i-2]$

return $t[n]$

$$T(n) = T(n-1) + T(n-2) > 2 \cdot T(n-2)$$

$$(T(n-1) \geq T(n-2)) > 2^2 \cdot T(n-4)$$

$$> 2^3 \cdot T(n-6)$$

$$\vdots$$

$$> 2^{n/2} T(1)$$

천만 분의 1초도 안 걸림

- 수열

- 팩토리얼

$$n! = 1 \cdot 2 \cdot 3 \cdots n$$

$$\begin{aligned} n! &= n \cdot (n-1) \cdot (n-2) \cdots 1 \\ &= n \cdot (n-1)! \end{aligned} \quad \leftarrow \text{재귀적 구조}$$

$$\text{factorial}(n) = \begin{cases} 1 & \text{if } n = 0 \\ n * \text{factorial}(n-1) & \text{if } n > 0 \end{cases}$$

- 수열

- 팩토리얼

```
fact(n) :  
    tmp ← 1  
    for i ← 2 to n  
        tmp ← i * tmp  
    return tmp
```

← 비재귀

```
fact(n) :  
    if (n = 0) return 1  
    else return n * fact(n-1)
```

← 재귀

$\Theta(n)$

■ 하노이 탑

- 원반 n 개, 기둥 3개 (a, b, c)
- 목표: n 개의 원반을 기둥 a로부터 기둥 b로 옮긴다

$n/(n+)$

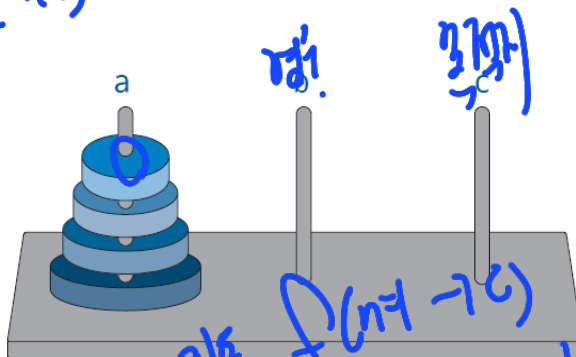


그림 2-2 원반이 4개인 하노이 탑 문제

- 원반은 한 번에 하나씩 옮길 수 있다.
- 큰 원반이 작은 원반 아래에 놓여 있어야 한다.

1 → 이동 $f(n-1 \rightarrow c)$
 2 → $f(n-1 \rightarrow c)$ $f(n-1 \rightarrow a)$
 3 → $f(n-2 \rightarrow a)$
 $f(n-1 \rightarrow c)$

$\text{move}(n, \text{start}, \text{temp}, \text{end})$
 $= \text{move}(n-1, \text{start}, \text{end}, \text{temp})$
 $\cdot \text{move}(1, \text{start}, \text{temp}, \text{end})$
 $\text{move}(n-1, \text{temp}, \text{start}, \text{end})$

■ 하노이 탑

- 함수 정의: $\text{move}(n, \text{source}, \text{destination}, \text{spare})$
 - n 개의 원반을 source 기둥으로부터 destination 기둥으로 옮긴다.
 - Spare : 보조 기둥

알고리즘 2-4 하노이 탑

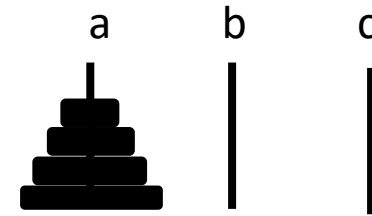
◀ 기둥 a에 있는 n 개의 원반을 기둥 c를 보조 기둥으로 사용해 기둥 b로 옮긴다.

$\text{move}(n, a, b, c)$:

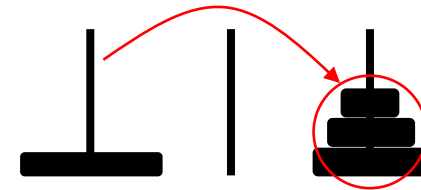
- ① if ($n > 0$)
 - ② $\text{move}(n-1, a, c, b)$
 - ③ a에 있는 원반을 b로 옮긴다
 - ④ $\text{move}(n-1, c, b, a)$

- 하노이 탑

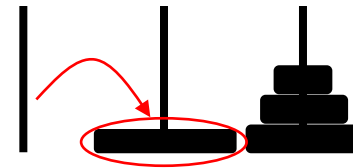
- `move(4, a, b, c)`



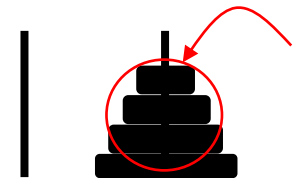
- `move(3, a, c, b)`



- a에 있는 (유일한) 원반을 b로 옮긴다

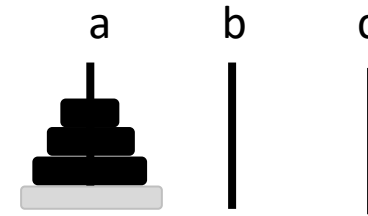


- `move(3, c, b, a)`

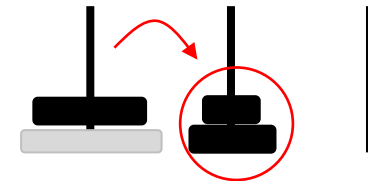


- 하노이 탑

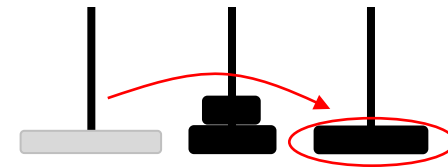
- move(3, a, c, b)



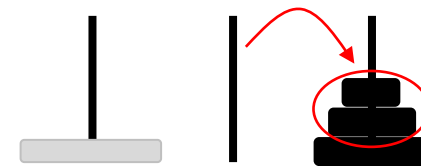
- move(2, a, b, c)



- a에 있는 원반을 c로 옮긴다



- move(2, b, c, a)



■ 마지막 직전 단계

알고리즘 2-4 하노이 탑

◀ 기둥 a에 있는 n개의 원반을 기둥 c를 보조 기둥으로 사용해 기둥 b로 옮긴다.

move(n, a, b, c):

- ① if ($n > 0$)
 - ② move($n-1$, a, c, b)
 - ③ a에 있는 원반을 b로 옮긴다
 - ④ move($n-1$, c, b, a)

move(1, x, y, z):

- ① if ($1 > 0$)
 - ② move(0, x, z, y)
x에 있는 원반을 y로 옮긴다
 - ④ move(0, z, y, x)

■ 선택 정렬

- 배열 $A[0, \dots, n-1]$ 의 n 개의 원소
- 목표: n 개의 원소를 크기순으로 정렬

알고리즘 2-6 선택 정렬(재귀 버전)

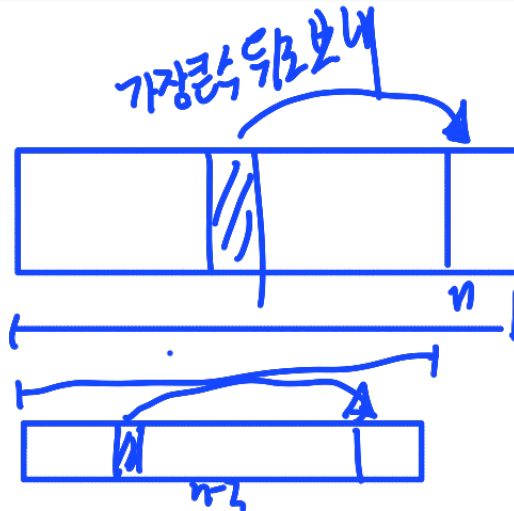
`selectionSort(A[], n):` ◀ 배열 $A[0 \dots n-1]$ 을 정렬한다

❶ if ($n > 1$)

❷ $A[0 \dots \text{last}]$ 중 가장 큰 수 $A[k]$ 를 찾는다

❸ $A[k] \leftrightarrow A[\text{last}]$ ◀ $A[k]$ 와 $A[\text{last}]$ 의 값을 교환한다

❹ `selectionSort(A, n-1)` ◀ 배열 $A[0 \dots n-2]$ 를 정렬한다



■ 선택 정렬

■ SelectionSort(A,n)

– 최대 원소를 찾는다.

29	10	14	37	13
----	----	----	----	----

– 최대 원소와 맨 오른쪽 자리 원소를 바꾼다.

29	10	14	37	13
----	----	----	----	----

– 맨 오른쪽 자리를 관심에서 제외한다.

29	10	14	13	37
----	----	----	----	----

– 원소가 1개 남을 때 까지 위를 반복한다.

29	10	14	13	37
----	----	----	----	----

■ 선택 정렬

■ SelectionSort(A,n)

- 최대 원소를 찾는다.
- 최대 원소와 맨 오른쪽 자리 원소를 바꾼다.
- 맨 오른쪽 자리를 관심에서 제외한다.
- 원소가 1개 남을 때 까지 위를 반복한다.

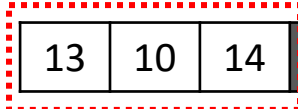
29	10	14	37	13
----	----	----	----	----

29	10	14	13	37
----	----	----	----	----

29	10	14	13	37
----	----	----	----	----



13	10	14	29	37
----	----	----	----	----



▪ 중위, 전위, 후위 표현법

- 수식을 표현할 때 연산자의 상대적 위치에 따라
 - 전위 표현 (prefix expression)
 - 중위 표현 (infix expression)
 - 후위 표현 (postfix expression)

로 나뉜다.

중위 표현법		전위 표현법	후위 표현법
$A + B * C - 2$	→	$- + A * B C 2$	$A B C * + 2 -$
$(A + B) * (C - 2)$	→	$* + A B - C 2$	$A B + C 2 - *$

그림 2-3 중위, 전위, 후위 표현법 예

■ 중위, 전위, 후위 표현법

(a) 중위 표현법

$\langle \text{infix} \rangle = \langle \text{변수} \rangle \mid \langle \text{infix} \rangle \langle \text{연산자} \rangle \langle \text{infix} \rangle$
 $\langle \text{연산자} \rangle = + \mid - \mid * \mid /$
 $\langle \text{변수} \rangle = A \mid B \mid \dots \mid Z$

일반적

3 + 2 * 5

(b) 전위 표현법

$\langle \text{prefix} \rangle = \langle \text{변수} \rangle \mid \langle \text{연산자} \rangle \langle \text{prefix} \rangle \langle \text{prefix} \rangle$
 $\langle \text{연산자} \rangle = + \mid - \mid * \mid /$
 $\langle \text{변수} \rangle = A \mid B \mid \dots \mid Z$

→
+ 3 * 2 5

더한다 3과 곱한다 2와

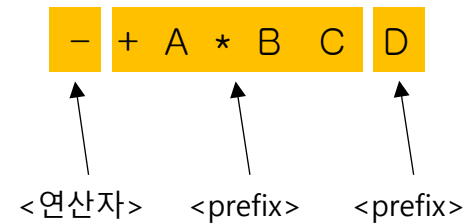
(c) 후위 표현법

$\langle \text{postfix} \rangle = \langle \text{변수} \rangle \mid \langle \text{postfix} \rangle \langle \text{postfix} \rangle \langle \text{연산자} \rangle$
 $\langle \text{연산자} \rangle = + \mid - \mid * \mid /$
 $\langle \text{변수} \rangle = A \mid B \mid \dots \mid Z$

←
3 2 5 * +

더한다 (곱한다 5와 2) 와 3

그림 2-4 재귀가 포함된 중위, 전위, 후위 표현법의 형식 언어



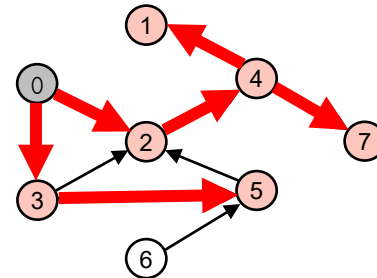
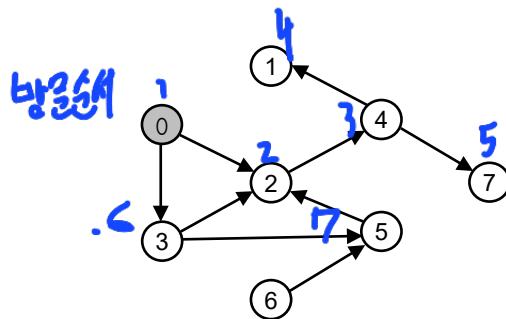
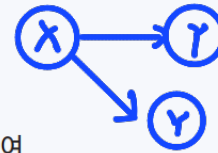
- 깊이 우선 탐색 (DFS) **DFS 또한 재귀의 형태**
 - 목표 : 노드 x에서 이룰 수 있는 모든 노드 찾기.

알고리즘 2-7 DFS 알고리즘

- ◀ 노드 x에서 이룰 수 있는 모든 노드 찾기
- ◀ 모든 노드 v에 대해 v.visited는 false로 초기화됨

DFS(x):

- 1 x.visited $\leftarrow$ true
x에서 화살표로 연결된 모든 노드 y 각각에 대하여
- 2 if (y.visited = false) DFS(y)



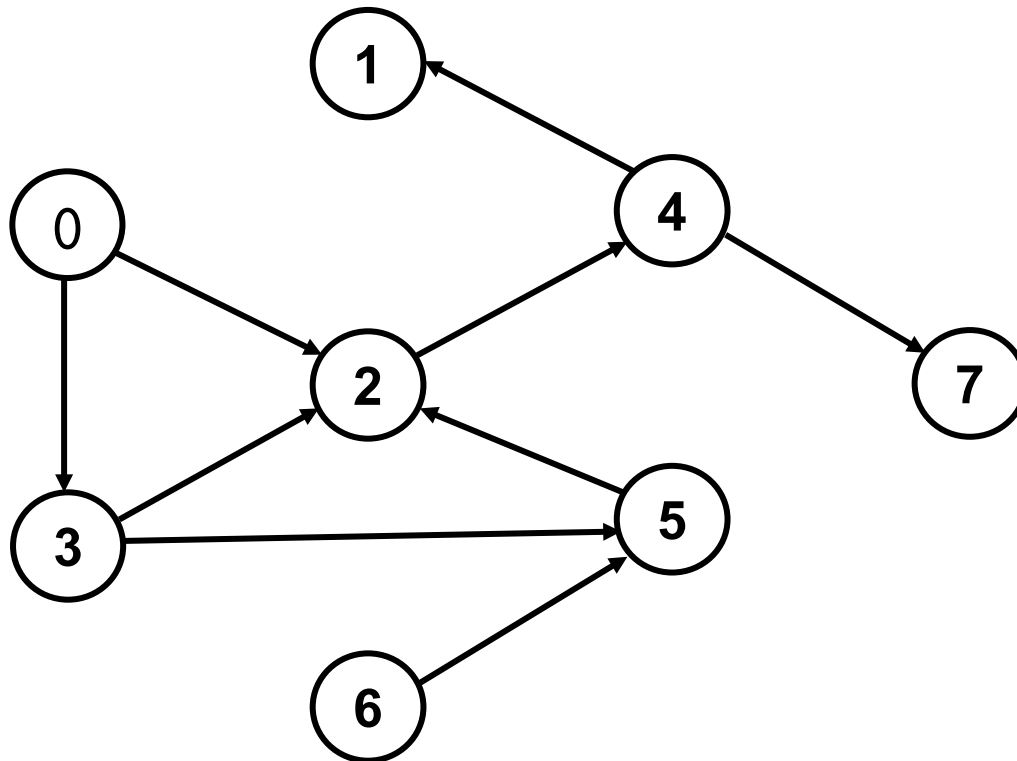
DFS(x)

=

{1,2,3,4,5,7}

DFS(0) : ① 0 방문
② 0이 가리키는 곳을 체크 DFS(0 → ?)

- 깊이 우선 탐색 (DFS)



이진 탐색 (이미 정렬된 배열에 한정해서)

binarySearch ($A[], n, x$) :

◀ 정렬된 배열 $A[0 \dots n-1]$ 에서 원소 x 를 찾는다

원소 x 의 위치 탐색.

$low \leftarrow 0$

$high \leftarrow n - 1$

while ($low \leq high$)

$mid \leftarrow (low + high) / 2$

if ($A[mid] < x$) $low \leftarrow mid + 1$

else if ($A[mid] > x$) $high \leftarrow mid - 1$

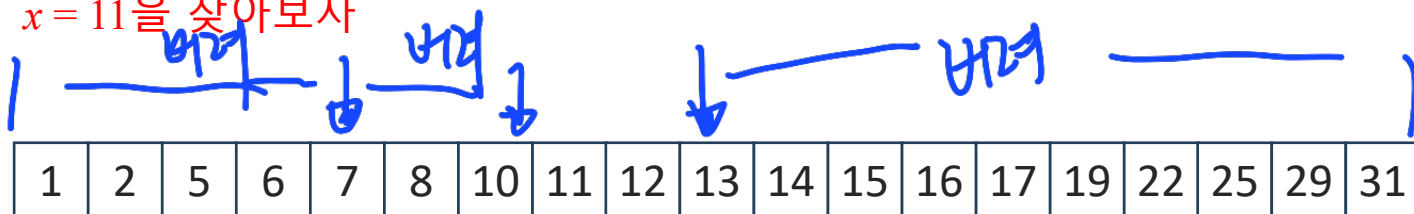
else return mid

return "Not found"

$A[mid] = x$



$x = 11$ 을 찾아보자



■ 이진 탐색

binarySearch (A[], x, low, high) :

◀ 정렬된 배열 A[low...high]에서 원소 x 를 찾는다

if (low > high) **return** “Not found”

mid $\leftarrow$ (low + high)/2

if (A[mid] < x) **return** binarySearch(A, x, mid+1, high)

else if (A[mid] > x) **return** binarySearch(A, x, low, mid-1)

else return mid

$x = 11$ 을 찾아보자

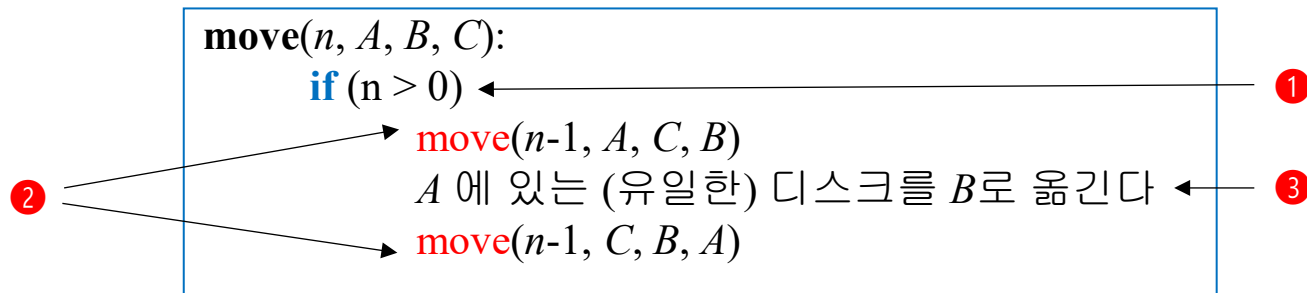
1	2	5	6	7	8	10	11	12	13	14	15	16	17	19	22	25	29	31
---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----

3. 재귀와 수학적 귀납법

- 재귀 알고리즘

- 재귀 알고리즘의 필수 조건

- 경계 조건 (또는 종료 조건) : 재귀 호출이 반복되다 궁극적으로 끝나는 조건
 - 재귀 호출
 - 관계 : 닮음꼴 작은 문제와 본 문제 간의 관계를 나타내는 부분



- 종료 조건이 없으면?

Base case가 없으면?

fib(n) :

```
if ( $n \leq 2$ ) return 1  
else return (fib( $n-1$ )+fib( $n-2$ ))
```

← base case

move(n, A, B, C) :

```
if ( $n > 0$ )
```

Base case가 없으면?

```
    move( $n-1, A, C, B$ )
```

A 에 있는 (유일한) 디스크를 B 로 옮긴다

```
    move( $n-1, C, B, A$ )
```

$T(n)$: 하노이 탑 알고리즘이 n 개의 원반을 옮기는 데 필요한 이동(원반 하나를 옮기는 것)의 총 횟수

$$T(n) = 2^n - 1$$

<증명: 수학적 귀납법>

i) (경계조건 만족) 우선 0개를 옮기는 데는 이동 횟수가 0번이다. $T(0) = 2^0 - 1 = 0$ 이므로 만족한다.

ii) (귀납적 가정) k 개의 원반을 옮기는 데 필요한 이동 횟수 $T(k) = 2^k - 1$ 이라 가정하자.

iii) (귀납적 전개: $k+1$ 개의 원반을 옮기는 데 필요한 이동 횟수 $T(k+1) = 2^{k+1} - 1$ 이 됨을 보인다) $k+1$ 개의 원반을 옮기는 작업은 원반 k 개를 옮기는 작업 두 번, 원반 1개를 옮기는 작업 한 번으로 구성되므로 다음이 성립한다.

$$T(k+1) = 2T(k) + 1 = 2(2^k - 1) + 1 = 2^{k+1} - 1$$

